

Task 3, Performance evaluation with Amdahl's Law

Unit: FIT3143 Parallel Computing, Semester 2 2026

Assessment: Lab 2 (Week 8), Message Passing Interface

Team: Erwyna Soo Wen Xin (36555789, esoo0013@student.monash.edu)

and Taabish Farooq Bhat (35473932, ttaa0006@student.monash.edu)

Task 3 by: Erwyna Soo Wen Xin

Supporting document for Task 3, submitted under the "Task 3 (Optional)" line of the submission checklist, which asks for any additional notes or documents that support the calculations and the experimental design. Every number here is produced by `run_benchmarks.sh`, `run_phases.py` and `make_graphs.py` on the machine below and is reproducible by re-running them. Some prose was drafted with AI assistance; see `AI_Declaration.md`.

1. Headline finding

The partitioning scheme in Task 1 does not distribute the work evenly, and the amount by which it fails is predictable in closed form.

`task1.c` gives rank r the numbers $3 + 2r + 2pk$. For any odd prime d dividing p , the term $2pk$ vanishes modulo d , so every number rank r ever tests is congruent to $3 + 2r$ modulo d . The rank whose class is $0 \bmod d$ therefore receives only multiples of d , and `is_prime` rejects those on its first loop iteration. One rank in d does effectively no work.

At $p = 3$, rank 0 spent 0.012 s and found 2 primes, while ranks 1 and 2 spent about 7.62 s each and found about 3.69 million each:

Rank	Search time (s)	Primes found
0	0.012	2
1	7.623	3,689,241
2	7.631	3,688,944

At $p = 4$, which has no odd factor, the same code splits perfectly:

Rank	Search time (s)	Primes found
0	3.831	1,844,654
1	3.829	1,844,589
2	3.827	1,844,669
3	3.831	1,844,275

The consequence is a saw tooth in every speedup curve. The code has, in effect, not p workers but

$p_{\text{eff}} = p * (1 - 1/d)$ d = smallest odd prime factor of p
 $p_{\text{eff}} = p$ when p is a power of two

and the measured imbalance ratio matches $1/(1 - 1/d)$ to two decimal places at every width we tested:

p	smallest odd factor d	measured comp max / comp mean	predicted $1/(1 - 1/d)$
1	none, power of two	1.00	1.00
2	none, power of two	1.00	1.00
3	3	1.50	1.50
4	none, power of two	1.00	1.00
5	5	1.25	1.25
6	3	1.50	1.50
7	7	1.17	1.17
8	none, power of two	1.00	1.00

9	3	1.50	1.50
10	5	1.26	1.25
11	11	1.14	1.10
12	3	1.52	1.50
13	13	1.11	1.08
14	7	1.30	1.17
16	none, power of two	1.04	1.00
20	5	1.29	1.25
24	3	1.53	1.50
28	7	1.21	1.17

This is why $p = 3$ is slower than $p = 2$ in results_by_procs.csv (8.52 s against 7.94 s), and why 2×7 beats 7×3 in the hybrid (2.29 s against 2.50 s) even though both command the same number of workers.

Recommended fix, not applied because Task 1 is Taabish's deliverable and we did not want to present code we had quietly changed: keep the cyclic idea but break the congruence, either by assigning interleaved chunks of candidates (the chunk of 1000 we used in Week 4 Task 2) instead of single strided values, or by having every rank skip multiples of 3 globally rather than one rank owning them. Either restores $p_{\text{eff}} = p$.

2. The machine and the measurement method

CPU	Apple M3 Max, 14 cores
MPI	Open MPI (Homebrew), single node, shared mem
Compiler	-O2 for all five programs, so the comparison
Fixed problem size	$n = 130,000,000$ (7,378,187 primes)
n sweep	31 values, 10,000,000 to 130,000,000
Width sweep	1 to 28, i.e. up to twice the core count
Repetitions	fastest of repeated runs, not the mean

Why fastest, not mean. A slow run means something else took a core. A fast run cannot be faster than the work genuinely takes. The minimum is the closest thing to a noise-free measurement available without a quiet machine.

Why the whole command is timed. Every headline run time in results_*.csv is the wall clock of the entire command as the shell sees it, mpirun included. The rubric asks for an overall speedup including communication, computation, sorting and file writing, and launching 14 processes is time a user genuinely waits for. Using the program's internal timer instead would hand Open MPI a free 0.184 s that the serial version never gets, that is the paired measurement from measure_launch.py, and it is a fixed cost that does not shrink with n , which is exactly why Open MPI loses to OpenMP at small n in Graph 1.

3. Experimental design: separating the serial and parallel parts

Amdahl's Law needs two numbers a single end-to-end stopwatch cannot give: the fraction of the work that can be spread out, and the fraction that cannot. We built three instrumented programs to get them.

File	What it is
serial_instr.c	Week 4 Task 1, unchanged, plus a timer around
task1_instr.c	Taabish's task1.c, unchanged, with per-phase
task2_instr.c	Taabish's task2.c, unchanged, with the threa

Four design decisions we would defend in Q&A:

(a) The instrument is separate from the thing measured. The MPI_Reduce calls that collect phase times cost time themselves. Leaving them in task1.c would make our submitted Task 1 slower than the code we are claiming.

(b) Phase times are the MAX across ranks, not rank 0's own. A collective finishes when the slowest rank arrives, so the maximum is what the wall clock feels. Rank 0's value would have hidden the imbalance in section 1 entirely.

(c) Every instrumented run opens with MPI_Barrier. This is the trap demonstrated in the Week 7 extra class, where an MPI_Recv appeared to take four seconds when it was really waiting for the sender to finish reading a file. Without a barrier, ranks clearing MPI_Init early start their clocks early and charge launch skew to "communication".

(d) A second MPI_Barrier sits between the search and the gather. This one was added after a first round of measurements reported gather times of over seven seconds, which was obviously not the cost of a shared-memory copy. Idle ranks were sitting inside MPI_Gather waiting for the busy ones, and that waiting was being billed as communication. The barrier drains the wait into its own phase, imbal_s. After the change, gather fell to 0.0051 s at $p = 1$ and 0.0361 s at $p = 28$, the real cost, while imbal_s carries the seconds. **Load imbalance and communication want completely different fixes, so a measurement that confuses them is worse than useless.**

Phases

Phase	Scales with p ?	Role
bcast, n out to every rank	grows slightly	communication, kappa
comp, the search loop	yes	parallel fraction, r_p
merge (hybrid only), thread buffers into one	grows with threads	charged to kappa
imbal, barrier wait after the search	grows with imbalance	diagnostic, not additive
gather, MPI_Gather + MPI_Gatherv	grows with p	communication, kappa
sort, qsort on the root	no	serial fraction, r_s
write, 7,378,187 lines of fprintf	no	serial fraction, r_s

imbal is deliberately not added into the total. The elapsed time from the start of the search to the barrier release is comp_max; for the fastest rank that is comp_min + imbal. Adding both would double count. Confirmed on the data: bcast + comp + gather + sort + write reproduces total_s at every width.

4. The model

$$S(p) = 1 / (r_s + r_p / p + \text{kappa}(p))$$

r_p = comp(1) / T(1) the search loop, the only part that spreads
 r_s = (sort + write)(1) / T(1) qsort and fprintf on the root, never spreads
 $\text{kappa}(p)$ = (bcast + gather)(p) / T(1) MEASURED at every p , not assumed

On the baseline. The fractions come from the $p = 1$ run of the parallel program itself, not from the Week 4 serial program. Taabish's Week 8 search uses a $6k \pm 1$ primality test while the Week 4 serial uses $d \pm 2$, so the Week 8 code is already about 1.4x faster per candidate before a single process is added (15.01 s against 21.26 s at the same n). Deriving r_p and r_s against the Week 4 program would fold that algorithmic win into what is meant to be a measurement of parallelism. The Week 4 comparison the specification asks for is still made in full; it is what Graphs 2, 3, 4 and 5 show. This section answers a different question.

On kappa. Most textbook treatments set kappa to zero. We measure it, following

the method from the Week 7 extra class. The finding is that on this hardware kappa is negligible: 0.00388 of the runtime even at $p = 28$. Every rank is on one machine, so MPI_Gatherv is a shared-memory copy rather than network traffic. Communication is not what stops the speedup here; the partition is. On CAAS across two compute nodes we would expect kappa to be orders of magnitude larger and to become the dominant term.

5. Measured parameters

Task 1, Open MPI

Parameter	Value
T(1), the whole program at one process	14.264 s
Parallel fraction r_p	0.9708
Serial fraction r_s	0.0288
Amdahl ceiling, $1 / r_s$ as p goes to infinit	34.68x
Phase split at $p = 1$	comp 13.847, sort 0.021, write 0.390
Phase split at $p = 14$	comp 1.893, gather 0.0166, sort 0.379, write

p	d	p_eff	comp max (s)	comp min (s)	kappa(p)	S Amdahl(p)	S Amdahl(p_eff)	S empirical
1	2^k	1.0	13.847	13.847	0.00036	1.00	1.00	1.00
2	2^k	2.0	7.019	7.015	0.00045	1.94	1.94	1.85
3	3	2.0	7.620	0.012	0.00059	2.83	1.94	1.71
4	2^k	4.0	3.817	3.812	0.00069	3.67	3.67	3.12
5	5	4.0	3.812	0.013	0.00069	4.47	3.67	3.10
6	3	4.0	3.838	0.006	0.00059	5.23	3.67	3.09
7	7	6.0	2.557	0.011	0.00068	5.95	5.23	4.28
8	2^k	8.0	1.934	1.931	0.00097	6.62	6.62	5.12
9	3	6.0	2.579	0.004	0.00079	7.27	5.22	4.24
10	5	8.0	1.976	0.006	0.00083	7.89	6.62	5.02
11	11	10.0	1.763	0.008	0.00107	8.46	7.87	5.50
12	3	8.0	2.240	0.003	0.00099	9.03	6.61	4.59
13	13	12.0	1.633	0.008	0.00121	9.55	9.01	5.78
14	7	12.0	1.893	0.005	0.00184	10.00	8.96	5.23
16	2^k	16.0	1.502	1.360	0.00138	11.00	11.00	5.99
20	5	16.0	1.485	0.003	0.00303	12.44	10.81	5.99
24	3	16.0	1.467	0.002	0.00397	13.65	10.70	6.01
28	7	24.0	1.485	0.003	0.00388	14.84	13.67	5.99

Task 2, hybrid Open MPI + OpenMP

Parameter	Value
T(1 worker)	15.163 s
Parallel fraction r_p	0.9726
Serial fraction r_s	0.0268

p x t	Width	p_eff x t	kappa	S Amdahl	S Amdahl(p_eff)	S empirical
1x1	1	1.0	0.00059	1.00	1.00	1.00
2x1	2	2.0	0.00048	1.95	1.95	1.87
2x2	4	4.0	0.00053	3.70	3.70	3.35
2x3	6	6.0	0.00053	5.28	5.28	4.64

3x2	6	4.0	0.00064	5.28	3.70	3.30
2x4	8	8.0	0.00051	6.72	6.72	5.77
2x6	12	12.0	0.00062	9.22	9.22	6.85
4x3	12	12.0	0.00066	9.21	9.21	6.69
2x7	14	14.0	0.00068	10.31	10.31	6.98
4x4	16	16.0	0.00071	11.32	11.32	6.80
7x3	21	18.0	0.00082	13.52	12.24	6.53
4x6	24	24.0	0.00056	14.72	14.72	6.86
7x4	28	24.0	0.00089	16.01	14.66	6.59

The hybrid partly escapes the residue problem. Only the MPI stride is congruence bound; inside a rank OpenMP hands out chunks with `schedule(dynamic, 1000)`, so a thread that draws a cheap chunk simply comes back for another. This is why the hybrid's best configurations use few processes and many threads.

6. Empirical results

Measurement	Value
Week 4 serial at $n = 130,000,000$	21.262 s
Best Open MPI speedup over the n sweep	7.80x at $n = 126,000,000$
Best Open MPI speedup over the width sweep	7.76x at $p = 11$
Best hybrid speedup	9.28x at 2×7
Best POSIX Threads speedup	7.79x
Best OpenMP speedup	8.27x
mpirun launch + MPI_Init overhead	0.184 s, fixed, independent of n
Serial write phase	0.466 s of 20.996 s total

Every parallel version plateaus around 7x rather than 14x. Two reasons, and we would give both: the M3 Max's 14 cores are 10 performance cores plus 4 efficiency cores, so the last four workers run at a fraction of the speed of the first ten; and past $p = 14$ there are more runnable processes than cores, so adding more only adds scheduling.

7. Answers to the questions the specification asks

How does the actual speedup compare against the theoretical speedup?
 Plain Amdahl at p overshoots badly, because it assumes the partition divides the work into p equal shares and it does not. Amdahl evaluated at p_{eff} reproduces the saw tooth in the measurement, which tells us the imbalance is the dominant error term rather than noise. The residual gap that p_{eff} still does not explain is core heterogeneity and, past $p = 14$, oversubscription, neither of which is in the model, and both of which are properties of the machine rather than the code.

Will more MPI processes always increase the speedup?
 No, and we can show three separate reasons from our own data. First, r_s is a hard floor: no number of processes makes the root's qsort or its 7,378,187 line file write any faster, capping us at 34.68x however wide we go. Second, $\kappa(p)$ grows with p , from 0.00036 at $p = 1$ to 0.00388 at $p = 28$, because every extra rank is another participant in the collective. Third and largest here, p_{eff} does not grow monotonically with p . Going from $p = 2$ to $p = 3$ adds a process but leaves the number of ranks doing real work at 2, so the run gets slower, not faster, the extra rank contributes nothing while still joining every collective. The same happens at every step onto a multiple of 3.

How would the workload distribution affect the speedup?

It is the dominant factor in this implementation, which is the whole of section 1. Cyclic partitioning was the right instinct, the cost of `is_prime(k)` grows like $\sqrt{k}$, so a block split would give the last rank the most expensive numbers and every other rank would wait for it. But a stride of exactly $2p$ locks each rank into a residue class, and that interacts with the trial-division fast rejection to idle one rank in d . The fix is to break the congruence, not to abandon cyclic.

Will the speedup results be the same across different machines?

No. r_s and κ are both properties of the machine, not the algorithm. A slower disk raises the write phase and therefore r_s , lowering the ceiling. Running across two computers over a network raises κ by orders of magnitude, because `MPI_Gatherv` stops being a shared memory copy and becomes real network traffic. A machine with homogeneous cores would push the plateau closer to 14x. The one thing that would transfer is the saw tooth, because it comes from arithmetic in the partitioning rather than from any hardware.

8. CAAS: the same code across two compute nodes

Everything above was measured on one laptop. On 9 September we ran the same programs on Monash's CAAS cluster (student-caas-headnode), with 8 MPI ranks split deliberately across two physical compute nodes, student-caas-n01 and student-caas-n02, so that `MPI_Gatherv` had to cross a network rather than copy within one machine. Job 39361, $n = 130,000,000$, output verified byte identical to the serial reference.

p	Total (s)	Speedup vs serial	Speedup vs p=1	Amdahl	Gap	kappa(p)
1	52.50	1.49x	1.00x	1.00	0.0%	0.00015
2	26.86	2.91x	1.95x	1.96	0.4%	0.00072
4	13.99	5.60x	3.75x	3.78	0.7%	0.00116
8	7.59	10.31x	6.91x	7.05	1.9%	0.00124

Serial baseline on the cluster: 78.29 s. Measured fractions: $r_p = 0.9819$, $r_s = 0.0179$, giving a ceiling of 55.9x.

Three things this changes

1. The code scales near linearly on hardware that suits it. 86.4% parallel efficiency at 8 processes, and the curve had not flattened when we stopped. On the laptop we plateaued near 7x. The difference is the machine, not the code: the M3 Max has 10 performance cores and 4 much slower efficiency cores, and past 14 workers we were oversubscribing. Cluster cores are homogeneous and dedicated.
2. Amdahl's Law predicts our measurement to within 2%. This is the result we would lead with. On the laptop, theory overshot measurement by roughly 23% at $p = 8$, and section 7 attributes that to core heterogeneity and oversubscription rather than to a flaw in the model. The cluster run tests that claim directly: on homogeneous dedicated cores with a balanced process count, the same model lands at 7.05 against a measured 6.91. The gap was the hardware, and we can now show it rather than argue it.
3. We predicted communication would dominate here. It does not. The earlier version of this document, and our closing slide, both said a CAAS run across two nodes was the missing measurement and that κ would finally become the dominant term. That prediction was wrong, and we would rather report it than quietly drop it.

The gather did become genuinely more expensive: 0.0633 s at $p = 8$ across the network against 0.0138 s at the same width on one machine, 4.6x larger, which confirms it really is network traffic now and not a memory copy. But as a

fraction of runtime κ is still only 0.0012. The reason is a ratio we had not thought about carefully enough: the payload is about 59 MB of primes, moved once, against 52 seconds of arithmetic. At roughly 0.9 GB/s of effective throughput the transfer is over before it matters. Communication would only dominate this problem at a much smaller n , or with far more ranks, or on a slower interconnect.

What this run does not show

We chose process counts of 1, 2, 4 and 8, all powers of two, so that the partition would be balanced and the scaling result would be clean. The consequence is that every measured imbalance ratio came back at 1.00, and the residue-class saw tooth from section 1 does not appear in this data at all. That finding still rests on the laptop sweep, which covered 18 widths including the ones with odd prime factors. Re-running on CAAS at $p = 3, 6$ or 12 would confirm it on the cluster, and we would expect it to, because it comes from arithmetic in the partitioning rather than from any property of the hardware.

Raw output: `caas/results/caas_results.txt`.

9. Limitations we would raise ourselves

1. The cluster run covers only balanced process counts. Section 8 used powers of two, so it confirms the scaling result but not the residue-class finding. That still rests on the laptop sweep.
2. The Week 4 baseline is a different primality test. Handled by reporting both baselines, but it is the first thing we would flag rather than be caught on.
3. `int` in the gather path. `MPI_Gatherv` takes `int` counts, so the code caps out above about 2 billion primes. Not reachable at our n , but a real limit.
4. κ is measured, not modelled. We report $\kappa(p)$ at the widths we ran rather than fitting a function, so the theoretical curve is only defined there.
5. The `p_eff` model is first order. It accounts for the rank that is fully idle, not for the fact that the surviving ranks also carry slightly uneven shares, which is why measured imbalance runs a little above $1/(1 - 1/d)$ at the larger widths.